TEORİK BİLGİSAYAR BİLİMLERİ · HESAPLAMA KURAMI · FİNAL ÖDEVİ

TURINGLAB

Öğrenci El Kitabı

~3 Haftalık Final Ödevi (4–22 Mayıs)

Tasarım · İnşa · Analiz · Sentez

DERS Hesaplama Kuramı · Bilgisayar Mühendisliği
BAŞLANGIÇ 4 Mayıs 2026 Pazartesi
SON TESLİM 22 Mayıs 2026 Cuma · 23:59
SÜRE 18 gün (~2.5 hafta) · Tek teslim noktası
ÇALIŞMA ŞEKLİ Bireysel · GitHub üzerinden teslim
TAHMİNİ İŞ YÜKÜ 25–30 saat (günlük ~1.5–2 saat)

Hazırlayan: Dr. Ali Çetinkaya · Selçuk Üniversitesi · Bilgisayar Mühendisliği
Değerlendirme: Ahmet Erharman

PROJE BAŞLANGICI · GİRİŞ

TuringLab Nedir?

4–22 Mayıs 2026 arasında, Turing makinelerini araştıran, tasarlayan ve çalıştırılabilir hale getiren tek bir Python projesi geliştireceksiniz: **TuringLab**. *Ödev üç zorunlu bölümden ve bonus bölümünden oluşur.*

Ödev Sonunda Elinizde Olacaklar

Ödev sonunda elinizde şunlar olacak:

- Tek-şeritli Turing makinelerini çalıştıran bir Python motoru
- 4 farklı problem için tasarlanmış Turing makineleri
- Çalışmanızı gösteren 5–8 dakikalık ekran kaydı videosu
- Tasarım kararlarınızı savunduğunuz kısa bir mini-rapor
- (Opsiyonel) çok-şeritli, NTM, görselleştirici gibi bonus özellikler

Bu artefakt, CS lisans öğrencisi olarak gösterebileceğiniz somut bir portfolyo parçasıdır. GitHub üzerinde public hale getirildiğinde mezuniyet sonrası iş başvurularında doğrudan referans verebileceğiniz bir çalışmadır.

Ödev Yapısı

BÖLÜM	TARİH	KONU	PUAN	ANAHTAR
Bölüm 1	4–10 Mayıs	TM Motoru	50	Biçimsel tanım → çalışan kod
Bölüm 2	11–17 Mayıs	4 TM Tasarımı	35	Algoritma → δ kuralları
Bölüm 3	18–22 Mayıs	Demo Video + Mini-Rapor	15	Çalışmayı sunma
Bonus	Esnek	MTM, NTM, görselleştirici, vb.	+20	İsteğe bağlı

TOPLAM: 100 puan zorunlu · +20 bonus · Maksimum 120 puan üzerinden değerlendirme.

TESLİM ALTYAPISI · GITHUB DİSİPLİNİ

GitHub Kuralları

Tüm proje teslimleri GitHub üzerinden yapılır. Bu sadece bir teslim aracı değil, aynı zamanda profesyonel yazılım geliştirme alışkanlığı edinmenizi sağlayan bir disiplindir.

1. Repo Kurulumu (Hafta 1, Cuma'ya kadar)

- GitHub'da **private** bir repo açın: `turinglab-{ad}{soyad}` (örnek: `turinglab-aliyilmaz`)
- Ahmet Erharman Hoca'ya okuma izni verin (collaborator olarak ekleyin)
- İlk commit: bu el kitabını `README.md` olarak repoya ekleyin, başına kendi adınızı yazın
- Repo URL'sini ders yönetim sistemine yapıştırın

2. Commit Disiplini

- **Düzenli commit:** 18 gün boyunca en az **10–15 anlamlı commit**. Hepsini son gün toplu commit yapanlar puan kaybeder. Commit history'niz çalışma sürecinizi gösterir.
- **Anlamlı mesajlar:** "fix" ✗ → "δ fonksiyonunda blank sembol bug'ı çözüldü" ✓
- **Tek teslim noktası:** Son tarih **22 Mayıs 2026 Cuma 23:59**. Bu tarihte `final` adlı bir tag/release oluşturun. Ahmet Erharman Hoca bu tag'in commit hash'i üzerinden değerlendirme yapacaktır.
- **Son teslim sonrası:** Repository'de değişiklik yapmayın. Yapılan değişiklikler değerlendirmeyi olumsuz etkileyebilir.

3. Hedef Repo Yapısı

Aşağıdaki yapı son teslimde hedeflenen organizasyondur. Bonus dosyalar (`multi_tape.py`, `ntm.py`, `visualizer.py`) yalnızca o bonus'u yapmışsanız bulunmalıdır:

```
turinglab/ ├── README.md # Proje tanıtımı, kullanım, demo video linki ├── REPORT.md # Mini-rapor (Bölüm 3) ├── requirements.txt # Python bağımlılıkları ├── .gitignore # __pycache__, venv, vs. ├── turinglab/ # Ana paket │ ├── __init__.py │ ├── tm_engine.py # Bölüm 1: TM motoru │ ├── multi_tape.py # (Bonus A) çok-şeritli motor │ ├── ntm.py # (Bonus B) non-deterministic motor │ └── visualizer.py # (Bonus D) görselleştirme ├── machines/ # Bölüm 2: Tasarlanmış TM'ler │ ├── unary_to_binary.yaml │ ├── binary_compare.yaml │ ├── string_copy.yaml │ └── student_choice.yaml ├── tests/ # pytest test dosyaları │ ├── test_tm_engine.py │ └── test_machines.py └── docs/ ├── design_notes.md # Tasarım kararları ├── demo_video.mp4 # Ekran kaydı VEYA README'de YouTube linki └── images/ # (Bonus D) görselleştirme çıktıları
```

PROJE STANDARTLARI · TEKNİK VE AKADEMİK

Genel Beklentiler

Teknik Standartlar

- **Dil:** Python 3.10+ (zorunlu — başka dil kabul edilmez)
- **Kod stili:** PEP 8. Bonus: `mypy` ve `ruff` ile linting (puan getirir)
- **Test:** `pytest`. Her milestone'da minimum test kapsamı belirtilmiştir
- **Bağımlılıklar:** Sadece izin verilen kütüphaneler (her milestone'da listelenmiştir)
- **Dokümantasyon:** Her public fonksiyona docstring (Google veya NumPy stili)

Akademik Standartlar

- **Tek başına çalışma:** Kod paylaşımı yasak. Tartışma serbest, kod kopyalama değil.
- **Kopya tespiti:** Otomatik benzerlik araçları (MOSS) çalıştırılır. Ayrıca commit history incelenir: tek seferlik 800 satırlık commit'ler ciddi şüphe yaratır.
- **Geç teslim:** Son tarih 22 Mayıs 2026 Cuma 23:59. Geç teslim her gün -10 puan (maksimum 3 gün, sonrasında ödev sıfırlanır).
- **Sözlü savunma (viva):** Şüpheli durumlarda Ahmet Erharman Hoca, öğrenciden kodun belirli bölümlerini açıklamasını isteyebilir. Açıklayamamak puan kaybına yol açar.

> ⚐ **KRİTİK NOT — DEĞERLENDİRME**
>
> *Ödev Ahmet Erharman Hoca tarafından değerlendirilecektir. Son teslim sonrası 2 hafta içinde geri bildirim alacaksınız. Sorularınız için ders yönetim sistemindeki Q&A; bölümünü kullanın.*

Final Notu Hesaplaması

BÖLÜM	KONU	PUAN
Bölüm 1	TM Motoru	50 puan
Bölüm 2	4 TM Tasarımı	35 puan
Bölüm 3	Demo Video + Mini-Rapor	15 puan
	Zorunlu Toplam	**100**
Bonus	Multi-tape, NTM, vb.	+20 puan
	Maksimum	**120**

SON TESLİM TARİHİ: 22 Mayıs 2026 Cuma · 23:59

BÖLÜM · 4–10 MAYIS · 50 PUAN

B1

TM Motoru

Deterministic single-tape Turing makinelerini çalıştıran temel Python kütüphanesini sıfırdan inşa et.

Bu bölüm ödevin en büyük kısmı (50 puan). Sonraki bölümler bu motoru kullanacak — burada yapacağınız her tasarım kararı, sonraki bölümler boyunca sizinle yaşayacak.

BÖLÜM 1 · TM MOTORU

Hedef ve Spesifikasyon

Hedef: *Deterministic single-tape Turing makinelerini çalıştıran bir Python kütüphanesi yazın. Sadece terminal arayüzü, sadece tek dosya (*`tm_engine.py`*), iki ana sınıf:* `TuringMachine` *ve* `Tape`.

Girdi Formatı (YAML)

Turing makineleri YAML dosyaları olarak tanımlanacak. Aşağıda *binary_increment* makinesinin (ikili sayıyı bir artıran) tam YAML örneği verilmiştir:

```
name: "binary_increment" description: "Binary sayıyı bir artırır. Örnek: 1011 -> 1100"
states: [q0, q_back, q_carry, q_done, q_accept] input_alphabet: ["0", "1"] tape_alphabet:
["0", "1", "B"] blank: "B" start_state: q0 accept_states: [q_accept] reject_states: []
transitions: - {state: q0, read: "0", next: q0, write: "0", move: R} - {state: q0, read:
"1", next: q0, write: "1", move: R} - {state: q0, read: "B", next: q_back, write: "B", move:
L} - {state: q_back, read: "0", next: q_done, write: "1", move: L} - {state: q_back, read:
"1", next: q_back, write: "0", move: L} - {state: q_back, read: "B", next: q_done, write:
"1", move: L} - {state: q_done, read: "0", next: q_done, write: "0", move: L} - {state:
q_done, read: "1", next: q_done, write: "1", move: L} - {state: q_done, read: "B", next:
q_accept, write: "B", move: R}
```

Python API (Zorunlu)

Aşağıdaki API'nin tamamı çalışmalıdır. Test betiklerimiz tam olarak bu sözdizimi ile çağıracak:

```
from turinglab import SingleTapeTM, RunResult # Yükleme tm =
SingleTapeTM.from_yaml("machines/binary_increment.yaml") # Çalıştırma result:
RunResult = tm.run( input_string="1011", max_steps=1000, verbose=False ) # Sonuç
incelemesi assert result.accepted is True assert result.final_tape.strip("B") ==
"1100" assert result.steps == 23 # örnek assert len(result.history) == 24 # Tek bir
adımı incelemek config = result.history[5] print(config.state, config.tape,
config.head_position)
```

BÖLÜM 1 · ZORUNLU DAVRANIŞLAR

Davranış ve Çıktı Spesifikasyonu

Zorunlu Durum Yönetimi

DURUM	BEKLENEN DAVRANIŞ
Geçerli δ kuralı yok	`accepted=False, reason="no_transition"`
`accept_states`'e ulaşıldı	`accepted=True, reason="accept"`
`max_steps` aşıldı	`accepted=False, reason="timeout"`
Kafa sola taştı (`head_position < 0`)	Karar size ait — README'de belirtin
Hatalı YAML	`ValueError` + anlamlı mesaj

Verbose Mod Çıktı Formatı

`verbose=True` modunda her adım, kafa konumu köşeli parantez içinde olacak şekilde basılmalıdır:

```
Adım 0 | Durum: q0 | Şerit: [1]011B | Hareket: R Adım 1 | Durum: q0 | Şerit: 1[0]11B |
Hareket: R Adım 2 | Durum: q0 | Şerit: 10[1]1B | Hareket: R Adım 3 | Durum: q0 | Şerit:
101[1]B | Hareket: R Adım 4 | Durum: q0 | Şerit: 1011[B] | Hareket: L Adım 5 | Durum: q_back
| Şerit: 101[1]0 | Hareket: L ...
```

Test Verileri

Aşağıdaki örnek TM'lerin YAML'larını ders sayfasında bulabilirsiniz: `unary_increment.yaml`, `even_a.yaml`, `binary_palindrome.yaml`. Bu makineleri test girdileriyle birlikte kullanarak motorunuzu doğrulayabilirsiniz.

Test Kapsamı (Zorunlu)

- `tests/test_tm_engine.py` içinde en az **8 test fonksiyonu**
- 3 farklı TM'in 5'er girdi için doğru çalışması
- Timeout durumu testi
- Hatalı YAML için `ValueError` testi
- `verbose=True` modu çıktı yakalama testi

İzinli Bağımlılıklar (Bölüm 1)

Sadece `PyYAML` (YAML parsing) ve `pytest` (test) kullanabilirsiniz. Başka kütüphane yok. Standart kütüphane (`typing`, `dataclasses`, vs.) serbesttir.

BÖLÜM 1 · DEĞERLENDİRME VE TUZAKLAR

Rubrik ve Sıkça Yapılan Hatalar

Değerlendirme Rubriği — Bölüm 1 (50 puan)

KRİTER	PUAN
YAML parser doğru çalışıyor, hatalı YAML için anlamlı hata	8
`run()` 3 verilen örnek TM için doğru sonuç	12
`result.history` her adımı doğru kaydediyor	6
Timeout, no_transition gibi kenar durumları doğru	6
`verbose` çıktısı şartnameye uygun ve okunabilir	6
Test dosyası 8+ test içeriyor ve hepsi geçiyor	6
Kod kalitesi: docstring, anlamlı isim, modülerlik	4
README'de kullanım örneği var, çalışıyor	2

⚐ TESLİM

Bölüm 1 için ayrı bir teslim yok — tek son teslim 22 Mayıs 2026 Cuma 23:59. Bu bölümü 10 Mayıs Pazar gününe kadar bitirmeniz önerilir, çünkü Bölüm 2 motoru kullanır.

Sıkça Yapılan Hatalar

Aşağıdaki hatalar her dönem öğrencileri saatlerce uğraştırır. Önceden bilirseniz tuzağa düşmezsiniz:

⚠ Şeridi `str` olarak tutmak

Python'da string immutable'dır — her yazma işlemi yeni string yaratır. Yavaş ve buggy. **Çözüm:** `list[str]` veya `dict[int, str]` (sparse).

⚠ Kafa pozisyonu için negatif index

`tape[-1]` Python'da listenin sonu demek, sola taşma değil! Ya `head_position < 0` durumunu açıkça kontrol edin, ya da `collections.deque` kullanın.

⚠ Test girdilerini sadece "çalışan" durumlarla sınırlandırmak

Hata yapmayan TM'iniz iyi değildir; hata yapması gereken durumda doğru hata veren TM iyidir. Ret durumlarını da test edin.

⚠ Blank sembol olarak boşluk seçmek

`blank: " "` seçerseniz debug etmek imkansız hale gelir. `B` veya `_` gibi görünür bir karakter kullanın.

⚠ Düzensiz commit alışkanlığı

Son gün toplu commit eden öğrenciler iki şey kaybeder: (1) commit history puanı, (2) bug çıktığında geri dönecek nokta yok. Her gün küçük commit yapın.

BÖLÜM · 11–17 MAYIS · 35 PUAN

B2

Tasarım Atölyesi

Kendi yazdığın motoru kullanarak 4 farklı problemin TM çözümünü tasarla.

Algoritmadan δ kurallarına çevirme — bu becerinin kendisi hesaplama kuramı eğitiminin temel kazanımıdır. Ne kadar çok TM tasarlarsanız o kadar derin anlarsınız.

BÖLÜM 2 · TASARIM ATÖLYESİ

4 TM, 4 Hikaye

Hedef: *Kendi yazdığınız motoru (M1) kullanarak 4 farklı problemin TM çözümünü tasarlayın. Her çözüm bir YAML dosyası + onu test eden bir Python test fonksiyonu içerir.*

Tasarlanacak Makineler — Zorunlu (3 adet)

TM-1 · Unary → Binary Çevirici

Girdi: `111` (3 sayısının tekli/unary gösterimi)
Çıktı (şerit sonu): `11` (3 sayısının ikili gösterimi)
Şerit alfabesi: `{1, 0, B, X}` (X = işaretleyici olarak kullanabilirsiniz)

TM-2 · İki İkili Sayıyı Karşılaştıran TM

Girdi: `1011#1100` (ayraç # ile)
Kabul: birinci sayı ikincisinden büyükse
Ret: değilse (eşit veya küçük)

TM-3 · Dizgi Kopyalayıcı

Girdi: `abba`
Çıktı (şerit sonu): `abba#abba`
Şerit alfabesi: `{a, b, B, #, A, B'}` (büyük A, B' işaretleyici olarak)

Tasarlanacak Makineler — Öğrenci Seçimi (1 adet)

TM-4 — Sizin Tasarımınız: Aşağıdaki seçeneklerden bir tane seçin (veya kendi öneriyle Ahmet Hoca'dan `design_notes.md` üzerinden onay alın):

- **(a)** 4'e bölünebilirlik testi (binary girdi)
- **(b)** İki dizginin anagram olup olmadığı testi
- **(c)** Basit parantez denge kontrolü: `(()(()))` kabul, `(()` ret
- **(d)** ROT-1 benzeri basit şifreleme: girdiyi alıp her harfi 1 öteleyen TM (a→b, b→c, ...)

> ⚐ **PEDAGOJİK NOT**
>
> *TM-2 (karşılaştırma) tek şeritte yapmak çok ileri-geri tarama gerektirir. Bu, M3'te göreceğiniz çok-şeritli TM motivasyonunu doğal olarak hazırlar — "Bu kadar kafa hareketi yerine 2 şerit kullansaydım daha kolay olurdu" diye düşünmeye başlayacaksınız.*

BÖLÜM 2 · ÇIKTILAR VE DEĞERLENDİRME

Beklenen Çıktılar ve Rubrik

Her TM İçin Beklenen Dosyalar

```
machines/ ├── unary_to_binary.yaml ├── binary_compare.yaml ├── string_copy.yaml └── student_choice.yaml docs/ └── design_notes.md # Her TM için 1-2 paragraf tests/ └── test_machines.py # her TM için 5+ girdi-beklenen çıktı testi
```

design_notes.md İçeriği

Her TM için aşağıdaki 5 soruyu yanıtlayın (~150–200 kelime). Bu kısım ödevin sadece "çalıştı/çalışmadı"yı değil, *düşünme sürecini* değerlendiren kısmıdır:

- **1. Strateji:** TM'in yüksek seviyeli algoritması nedir? (Doğal dilde)
- **2. Durum sayısı:** Kaç durum kullandınız? Neden? Daha az durum mümkün müydü?
- **3. Şerit alfabesi seçimi:** Neden bu yardımcı semboller? Alternatif var mıydı?
- **4. Karmaşıklık:** Girdi uzunluğu n iken kaç adımda biter? (Big-O)
- **5. Hata ayıklama hikayesi:** En zorlandığınız bug ne oldu, nasıl çözdünüz?

> ⚐ **DİKKAT**
>
> *5. madde önemli — "her şey tıkır tıkır gitti" yazan öğrencilerden değerlendirme sırasında şüphelenilir. Yaptığınız hataları ve onları nasıl çözdüğünüzü dürüstçe yazın. Bu, kodun gerçekten size ait olduğunu gösteren en güçlü kanıttır.*

Test Kapsamı

Her TM için en az 5 test girdisi:

- 2 kabul edilmesi gereken girdi
- 2 reddedilmesi gereken girdi
- 1 kenar durum (boş girdi, tek karakter, vb.)

Değerlendirme Rubriği — Bölüm 2 (35 puan)

KRİTER	PUAN
TM-1 çalışıyor, tüm testleri geçiyor	6
TM-2 çalışıyor, tüm testleri geçiyor	6
TM-3 çalışıyor, tüm testleri geçiyor	7
TM-4 (öğrenci seçimi) çalışıyor, makul derecede ilginç	7

KRİTER	PUAN
`design_notes.md` her TM için yeterince derin	5
Test kalitesi (kenar durumlar dahil)	3
Commit disiplini (anlamlı mesajlar, düzenli aralıklarla)	1

ÖNERİ

Bölüm 2'yi 17 Mayıs Pazar gününe kadar bitirmeniz önerilir. Bu aşamada toplam puan 85/100 (Bölüm 1 + 2). Geri kalan 5 gün Bölüm 3 (demo + rapor) ve isterseniz bonus için.

BÖLÜM · 18–22 MAYIS · 15 PUAN

B3

Demo Video + Mini-Rapor

Çalışmanı bir ekran kaydı videosu ile sun. Tasarım kararlarını kısa bir mini-rapor ile savun.

Bu bölüm yazılım yetkinliğini iletişim becerisiyle birleştirir. Profesyonel hayatta kod kadar önemli olan iki şey: yaptığın işi anlatabilmek ve dokümante edebilmek.

BÖLÜM 3 · DEMO VİDEOSU

Ekran Kaydı Videosu

Hedef: *Çalışmanızı kayıt eden 5–8 dakikalık bir ekran kaydı (screen recording) videosu hazırlayın. Sınıfta canlı sunum yapmayacaksınız — bu video sunumun yerini alır.*

> ⚐ **NEDEN VİDEO?**
>
> *Ekran kaydı videosu canlı sunumdan üç açıdan üstündür: (1) sınırsız sayıda alıp en iyisini teslim edebilirsiniz, (2) Ahmet Hoca videoyu uygun zamanında izleyip detaylı not düşebilir, (3) ödev sonrasında portfolyonuzun parçası olur.*

Video Spesifikasyonları

SPESİFİKASYON	GEREKSİNİM
Süre	5–8 dakika (hedef: 6 dakika)
Format	MP4 (H.264 codec)
Çözünürlük	Minimum 1280×720 (HD)
Ses	Anlaşılır mikrofon kaydı zorunlu
Dil	Türkçe
Dosya boyutu	Maks. 200 MB (GitHub limit) *veya* YouTube unlisted linki
Konum	`docs/demo_video.mp4` *veya* README'de YouTube linki

İçerik Şablonu

1. Açılış (45 saniye)

Adınız, projenizin adı (TuringLab), bu video ne anlatacak — kısa özet.

2. Canlı Demo (2–3 dakika)

- Bir TM YAML dosyası açın, içeriğini gösterin
- Terminalde simülatörü çalıştırın, `verbose` modda adım adım çıktı düşürün
- Tasarladığınız 4 TM'den en az 2'sini çalışırken gösterin

3. En Sevdiğiniz Tasarım Kararı (1–2 dakika)

En zor olan kısım neydi? Nasıl çözdünüz? Alternatif yaklaşım var mıydı?

4. Bonus (Yaptıysanız) + Kapanış (1 dakika)

Eğer bonus bölümlerden yaptıysanız kısaca gösterin. Sonra: bir hafta daha olsa ne eklerdiniz, bu projeden öğrendiğiniz en önemli şey ne?

BÖLÜM 3 · KAYIT VE MİNİ-RAPOR

Kayıt Araçları ve Mini-Rapor

Önerilen Ekran Kaydı Araçları

ARAÇ	AÇIKLAMA	PLATFORM
OBS Studio	Açık kaynak, ücretsiz	Tüm platformlar
ShareX	Ücretsiz, hızlı kayıt	Windows
QuickTime	Yerleşik, kolay	macOS
Loom	Web tabanlı, ücretsiz tier	Tüm platformlar

Kayıt İpuçları

- **Önce script yazın.** Doğaçlama yapmayın — yarım sayfalık bir senaryo hazırlayın.
- **Sessiz oda seçin.** Klima, fan, dış sokak gürültüsü kaydı bozar.
- **Birkaç deneme yapın.** İlk kayıt asla en iyi değildir.
- **Yazı tipini büyütün.** Editör ve terminal yazı tipini en az 14pt yapın.
- **Akıcı konuşun.** "Eee" gibi dolgu kelimelerden kaçının.

Mini-Rapor (REPORT.md)

3–5 sayfa *Markdown formatında bir rapor. Bu kısa rapor projenizin teknik dokümantasyonudur.*

İçerik Şablonu

- **1. Giriş (~yarım sayfa)** — TuringLab nedir, ne yapar.
- **2. Mimari (~1 sayfa)** — Modüllerin organizasyonu, önemli tasarım kararları (şeridi nasıl temsil ettiniz, neden, vb.).
- **3. Tasarlanan TM'ler (~1 sayfa)** — 4 makineyi kısaca özetleyin, en zoru hangisiydi neden.
- **4. Kavramsal Tartışma (~1 sayfa)** — Aşağıdaki sorulardan **1 tanesini** seçin (~250 kelime):
  - (a) Halting problemini TuringLab içinde "çözmek" mümkün mü? Neden değil?
  - (b) Bir TM'in dilini başka bir TM tarafından tanımak (UTM!) için TuringLab nasıl genişletilebilir?
  - (c) Modern bir programlama dili (Python) ile TM arasındaki "boşluk" nedir?

- **5. Sınırlar ve İleri Çalışma (~yarım sayfa)** — Eksik kalan ne, bir hafta daha olsa ne eklerdiniz.
- **6. Kaynakça** — Kullandığınız referansları belirtin.

BÖLÜM 3 · DEĞERLENDİRME

Rubrik — Bölüm 3 (15 puan)

KRİTER	PUAN
Demo videosu spesifikasyona uygun (süre, format, ses kalitesi)	3
Demo videosu içeriği: canlı çalışan TM'ler gösterilmiş	3
Demo videosu: tasarım kararları açıklanmış	2
Mini-rapor: Mimari ve tasarım kararları savunulmuş	3
Mini-rapor: Tasarlanan TM'ler bölümü	2
Mini-rapor: Kavramsal tartışma (1 soru, makul derinlik)	2

TESLİM

Demo videosu repoda veya YouTube unlisted linki olarak `README.md`*'de belirtilmiş olmalıdır. Mini-rapor* `REPORT.md` *olarak repo kökünde olmalı. Tek son teslim: 22 Mayıs 2026 Cuma 23:59.*

İSTEĞE BAĞLI · EK PUAN · +20 PUAN

BONUS

Daha İleri Git

Zorunlu bölümleri tamamladıktan sonra ek puan kazanmak isteyenler için. Hiçbir bonus zorunlu değildir; ama biri bile yapmak ders ortalamanızı yukarı çekebilir.

Bonus seçenekleri bağımsızdır — birini, ikisini veya hepsini yapabilirsiniz. Toplamda en fazla +20 bonus puan kazanabilirsiniz.

BONUS BÖLÜMÜ · EK PUAN SEÇENEKLERİ

4 Bonus Seçeneği

Aşağıdaki seçenekler bağımsızdır. Her biri için motoru genişleten ayrı bir Python modülü yazmanız gerekir. Demo videosunda bonus'larınızı kısaca gösterin; mini-rapora kısa bir bonus paragraf ekleyin.

Bonus A — Çok-Şeritli TM (+8 puan)

`turinglab/multi_tape.py` içinde `MultiTapeTM` sınıfı yazın. YAML'da `num_tapes: k` tanımlanabilmeli. `read`, `write`, `move` alanları k uzunluğunda liste olur.

- En az bir adet 3-şeritli TM tasarlayın (örn. ikili toplama)
- En az 5 test girdisi için doğru çalışmalı
- `tests/test_multi_tape.py` içinde testler

Bonus B — Non-Deterministic TM (+8 puan)

`turinglab/ntm.py` içinde `NondeterministicTM` sınıfı yazın. δ artık `choices` listesi döner. **BFS ile** hesaplama ağacını gezin (DFS değil — sonsuza takılır).

- En az bir NTM tasarlayın (örn. "01 alt-dizgisi var mı?")
- `max_depth` ve `max_branches` parametreleri zorunlu
- Kabul yolu döndüren `accepting_paths` alanı

Bonus C — Karşılaştırmalı Analiz (+4 puan)

Bonus A veya B yapmış olmak gerekir. Aynı dili (örn. `L = {w | w'de '01' var}`) tek-şeritli, çok-şeritli ve/veya NTM ile çözüp adım sayılarını karşılaştırın. matplotlib ile bir grafik çizin, `docs/comparison.png` olarak kaydedin. Mini-rapora 1 paragraflık yorum ekleyin.

Bonus D — Görselleştirici (+5 puan)

`turinglab/visualizer.py` yazın. Bir TM çalıştırması verildiğinde her adımın görsel temsilini PNG olarak üret. Şerit kutuları, kafa konumu, durum etiketi gözüksün. En az 5 PNG kareyi `docs/images/` altında commit'leyin.

- Kullanılabilir kütüphaneler: `Pillow`, `imageio`
- Bonus üstü bonus: bir GIF de üretirseniz +1 ek puan

⚐ STRATEJİ

Önce zorunlu 3 bölümü tamamlayın (100 puan hedefi). Vaktiniz kalırsa Bonus A veya B'den birini seçin — ikisi de en kolay olanları. Sonra Bonus C veya D ile devam edin. Hepsini yapmaya çalışıp hiçbirini tam yapamamak en kötü senaryodur.

PROJE PLANLAMASI · HAFTAYA HAFTA

Çalışma Takvimi · 4–22 Mayıs

Aşağıdaki takvim, 18 günlük ödev için önerilen plandır. Tek son teslim tarihi **22 Mayıs 2026 Cuma 23:59***'dur — ara teslimler yok. Bu plan kendinizi takip etmeniz için yol haritasıdır.*

HAFTA	TARİH	ANA KONU	YAPILACAKLAR
Hafta 1	4–10 Mayıs	Bölüm 1 — TM Motoru	YAML parser, run() fonksiyonu, history, testler. Hafta sonunda motor çalışıyor olmalı.
Hafta 2	11–17 Mayıs	Bölüm 2 — 4 TM Tasarımı	3 zorunlu + 1 seçim TM. Her biri için YAML + test + design_notes.md girişi.
Son Hafta	**18–22 Mayıs**	**Bölüm 3 + Bonus + TESLİM**	Demo videosu kaydı, mini-rapor (REPORT.md), README cilalama. **22 Mayıs Cuma 23:59** son teslim.

⚐ STRATEJİ İPUCU

Son hafta sadece 5 gün — bu yüzden Bölüm 3 (demo + rapor) için bu süre kısa görünebilir. Bölüm 2'yi 17 Mayıs'tan önce bitirmeye odaklanın; demo videosu + mini-rapor için 18-22 Mayıs arasında 3-4 saat yeterli olacaktır. Bonus yapacaksanız Hafta 2'de paralel başlayın.

Bölüm Bağımlılıkları

Bölümler birbirine bağlıdır. Bölüm 1'i (motor) bitirmeden Bölüm 2'ye (TM tasarımları) tam başlayamazsınız, çünkü TM'leri test etmek için motor gerekir.

```
Bölüm 1 (motor) ──→ Bölüm 2 (TM tasarımları) ──→ Bölüm 3 (demo + rapor) │ └──→ Bonus (paralel — istediğiniz zaman)
```

Geç Teslim Politikası

- İlk 24 saat: **−10 puan**

- İkinci 24 saat: **−20 puan**
- Üçüncü 24 saat: **−30 puan**
- Sonra: **0 puan**, ödev başarısız

Mazeretli durumlar: Sağlık vb. mazeretler için doktor raporu/resmi belge ile uzatma — ders yönetim sistemine başvuru yapın.

SIKÇA SORULAN SORULAR · TUZAKLAR

SSS ve Kapanış

Ödevde Python yerine başka bir dil kullanabilir miyim?

Hayır. Tüm sınıf aynı dilde çalışır — bu hem Ahmet Hoca'nın değerlendirmesini kolaylaştırır, hem de örnek paylaşmayı mümkün kılar. Python 3.10+ zorunlu.

TM motorunu GitHub'da hazır bulabilir miyim?

Evet, çok sayıda hazır TM simülatörü kütüphanesi GitHub'da mevcut. **Onları kullanmak yasak.** Otomatik benzerlik tespiti yapılır. Hazır kütüphaneye bakmak yerine konsept anlamak için *nasıl çalıştıklarını* okuyun.

Bonus yapmam gerekli mi? Sadece zorunlu bölümlerle 100 puan alabilir miyim?

Evet, hiçbir bonus zorunlu değildir. Bölüm 1+2+3 hatasız tamamlanırsa 100/100 puan alabilirsiniz. Bonus sadece ek puan getirir. Vaktiniz kalırsa bir bonus seçeneğine yönelmenizi öneririz.

Bonus'lar arasında hangisi en kolay?

Genellikle Bonus A (Multi-tape) — çünkü tek-şeritli motorun küçük bir uzantısıdır. Bonus B (NTM) BFS gerektirir, biraz daha düşünmek lazım. Bonus D (Görselleştirici) eğlencelidir ama Pillow öğrenmek zaman alır. Bonus C en küçük puanlı, sadece A veya B yaptıysanız mantıklı.

Demo videosunda yüzümü göstermem gerekli mi?

Hayır — video sadece ekran kaydı olabilir, ses anlaşılır olduğu sürece. Yüz görünümü isteğe bağlıdır.

GitHub'a ücretsiz hesap yetiyor mu?

Evet. Bu projede private repo gerekiyor — GitHub Free hesap, sınırsız private repo veriyor. Eğitim hesabı (GitHub Education) ile ek özellikler de gelir.

Kod yazma sürecinde Stack Overflow, dokümantasyon, Python rehberlerini kullanabilir miyim?

Evet — bunlar normal araştırma kaynaklarıdır. Ancak **başka birinin yazdığı tam çözümleri kopyalamak yasaktır.** Sınır basit: kavramı veya tekniği öğrenmek için kullanın, sonra kendi kodunuzu yazın. Şüpheli durumlarda referans verin (yorum satırında).

22 Mayıs Cuma'dan sonra repo'da değişiklik yapabilir miyim?

Hayır. Son teslim tarihinden sonraki commit'ler değerlendirmeyi olumsuz etkileyebilir. Cuma 23:59'da `final` tag'ini oluşturun ve repo'yu öyle bırakın.

Office Hours ve Soru Sorma

Sorularınızı önce ders yönetim sistemindeki Q&A; bölümünden sorgulayın — olasılıkla başka bir öğrenci aynı soruyu sormuştur. Cevap yoksa Ahmet Erharman Hoca'nın belirteceği iletişim kanalından doğrudan sorabilirsiniz.

Soru sorarken **tam bilgi verin**: "Çalışmıyor" demeyin — "Şu girdiyi verdim, şunu bekledim, şu çıktı, hatayı şurada düşünüyorum" deyin. Bu hem cevabı hızlandırır, hem de aslında çoğu zaman soruyu yazarken cevabı kendiniz bulursunuz.

SON SÖZ · BU PROJEDEN BEKLENEN

Bir Düşünsel Yatırım

TuringLab basit görünen bir ödev değildir. 4–22 Mayıs arasında yazacağınız bu Python paketi, bilgisayar biliminin temel sorularını — hesaplama nedir, sınırları nelerdir, varyantlar arası eşdeğerlik nasıl kanıtlanır — kendi parmaklarınızla deneyimlemeniz için tasarlandı.

Ödev bittiğinde elinizdeki şeyin değerini şöyle düşünün:

- **Akademik:** Hesaplama kuramı dersinin kavramlarını sözel olarak değil *somut olarak* anlamış olacaksınız. Yüksek lisans seviyesi karmaşıklık derslerine sağlam bir zeminde gireceksiniz.
- **Yazılım Mühendisliği:** Test yazma, refactoring, GitHub disiplini, dokümentasyon — hepsi profesyonel iş hayatında ilk günden ihtiyacınız olacak beceriler.
- **Portfolyo:** Çalışan, dokümante edilmiş, açık kaynak yapılabilen bir Python kütüphanesi. CV'nize "GitHub linki" olarak girer, mülakatlarda doğrudan referans verirsiniz.
- **Düşünme:** Soyut bir matematiksel modeli (Turing makinesi) somut bir mühendislik artefaktına çevirmek — bu süreç bilgisayar bilim eğitiminin özüdür.

> *"We can only see a short distance ahead, but we can see plenty there that needs to be done."*
>
> — Alan M. Turing, 1950

BAŞARILAR DİLERİZ.

Sorularınız için Ahmet Erharman Hoca ile iletişime geçebilirsiniz.

Hazırlayan: Dr. Ali Çetinkaya · Selçuk Üniversitesi · Bilgisayar Mühendisliği